

Constrained Generation of Theme-Based Hindi Dohas

Divyanshu Jain Kavan Gandhi Nikhil Patidar Abhinav Jha
2025201059 2025201078 2025201081 2024201008

Abstract

We present a system for constrained generation of Hindi dohas two-line poems governed by a strict 13+11+13+11 matra-based prosodic structure. Given a user-defined theme and context, our pipeline generates grammatically correct, semantically coherent dohas that adhere to the formal structural requirements of the poetic form. The system employs a custom 14.7M-parameter Encoder-Decoder Transformer trained in two stages: domain adaptation via span-corruption pre-training on 58,500 Hindi poems, followed by supervised fine-tuning using a novel dual-decoder architecture with gated cross-attention. A two-pass inference strategy resolves the train-inference distributional gap. Evaluated on 50 generated dohas, our model achieves an average matra count of 46.58 (theoretical ideal: 48), an External Novelty score of 0.9257, and an average human evaluation score of 3.57/5 across grammatical correctness, poetic soundness, and thematic coherence. All code, datasets, and model checkpoints are publicly available at: <https://github.com/KGan31/Doha-Gen>

1 Introduction

Large Language models are particularly effective at producing fluent and semantically coherent text in free form settings. However, generating text that must strictly conform to predefined structural or formal constraints remains a challenging problem. Poetry generation is a prominent example of such constrained text generation, where outputs must satisfy rules related to meter, line length, rhyme, and overall structure. Limited work has been done especially in poetry generation for Indic Languages.

This project addresses the identified gap by focusing on the generation of Hindi dohas, a

structurally rigid poetic form. Given a user-provided theme and context, our objective is to generate grammatically correct and semantically coherent dohas that adheres to the necessary constraints.

To achieve this, we (1) curate and annotate a specialized dataset of 7,889 high-quality Hindi dohas; (2) train a custom lightweight Transformer entirely from scratch using a two-stage pipeline combining span-corruption domain adaptation with a dual-decoder fine-tuning strategy; and (3) employ a two-pass inference procedure that conditions doha generation on a model-generated semantic meaning. Our key architectural contribution is a Doha Decoder with learnable gated cross-attention that fuses encoder representations with meaning-decoder hidden states, enabling simultaneous structural and semantic conditioning

2 Relevant Work

The doha is a traditional form of Hindi poetry popularized by saint-poets like Kabir and Rahim, defined by a specific matra-based prosodic structure. Unlike English poetry, which is often based on syllable stress, Hindi poetry follows a matra-based prosodic system, where a matra represents a unit of syllabic duration. In valid doha there are two lines, the first half of each line contains 13 matras, while the second half contains 11 matras, and both lines typically end in a rhyme, contributing to its musical and lyrical quality.

An empirical analysis of approximately 7,800 dohas indicates that while the 48-matra count is the standard, it is not universal; only 66% of the corpus adheres strictly to this total, with most outliers deviating by no more than three matras. This mixture of rigid theo-

retical constraints and observed rhythmic flexibility provides a complex, nuanced testbed for evaluating constrained text generation in Indic languages.

Chandomitra (Jagadeeshan et al., 2025) presents a structured dataset for translating English input into Sanskrit poetry that strictly adheres to the Anustubh meter, a classical Sanskrit metrical form. The work emphasizes the importance of explicitly modeling prosodic constraints to ensure metrical correctness in generated poetry. It demonstrates that unconstrained language models are insufficient for classical Indian poetic forms and motivates the use of hard constraints when dealing with matra or syllable governed meters.

The Mechanical Bard (Agnew et al., 2023) explores constrained poetry generation in English by generating Shakespearean sonnets using a pre-trained GPT-2 language model. The authors enforce a combination of hard constraints, such as syllable counts and rhyme schemes, and soft constraints, such as stylistic and poetic coherence, during the generation process.

Our proposed work draws inspiration from both of these studies. Similar to Chandomitra, Our focus is on theme-conditioned generation of Hindi dohas in Devanagari script, a relatively underexplored poetic form governed by matra-based constraints.

3 Dataset Details

3.1 Doha Dataset Details

Data Collection from KavitaKosh: We scrapped Hindi Poems and Hindi Dohas from *KavitaKosh* (KavitaKosh), one of the most comprehensive online repositories of Hindi poetry. We were able to collect 8,500 dohas and about 58 thousand Poems.

Data Cleaning and Preprocessing: Raw scraped data required multi-stage cleaning before use in model training.

We cleaned dataset performing exact-string deduplication and Unicode NFC normalization for encoding consistency. We then filtered the corpus to retain only valid two-line structures, discarding fragments or extra verses. Finally, the text was sanitized using regex rules to remove HTML artifacts, ASCII punctuation, and corrupt Devanagari sequences.

After cleaning, the final dataset comprised **7,889 high-quality dohas** distributed across **67 unique authors**.

Dataset Schema Three fields *Theme*, *Context*, and *Meaning* were generated by the Gemini API, while the remaining three fields *Author*, *Doha* and *URL* were obtained directly through scraping.

Dataset Statistics & Analysis: Table 1 lists the twelve most frequent thematic categories. The corpus is dominated by *Prem* (Love), *Samajik* (Social), *Bhakti* (Devotion), and *Niti* (Ethics), together accounting for over 34% of all entries. The five most prolific poets : Rasleen (911), Deenanath Sumitr (798), Bharatendu Mishra (698), Ranjana Verma (643), and Dharanidas (510) account for approximately 45% of the corpus.

Table 1: Top 12 Thematic Categories

Rank	Theme	Translation	Count
1	Prem	Love	809
2	Samajik	Social	699
3	Bhakti	Devotion	641
4	Niti	Ethics	605
5	Virah	Longing	352
6	Jivan	Life	339
7	Karun	Compassion	327
8	Shringar	Aesthetics	325
9	Saundarya	Beauty	320
10	Rajaniti	Politics	309
11	Prakrti	Nature	216
12	Riti	Tradition	204
–	<i>Others</i>	<i>92 remaining themes</i>	<i>2,743</i>
Total			7,889

Dataset Quality Considerations:

Since theme labels are generated by Gemini rather than manually curated, minor inconsistencies may exist (e.g., a doha assigned to both *Prem* and *Sringar* on different runs). Finally, the author-frequency distribution is skewed: five poets contribute nearly half of all entries, which may bias a fine-tuned model towards their stylistic characteristics.

3.2 Kavita Dataset Details

Our dataset comprises 58,530 poems across 2,904 unique poets. Our corpus contains a total of 11,733,644 words with a unique vocabulary size of 65,416 distinct words. Sentence-level analysis reveals an average of 47.48 words per sentence across 247,120 total sentences in

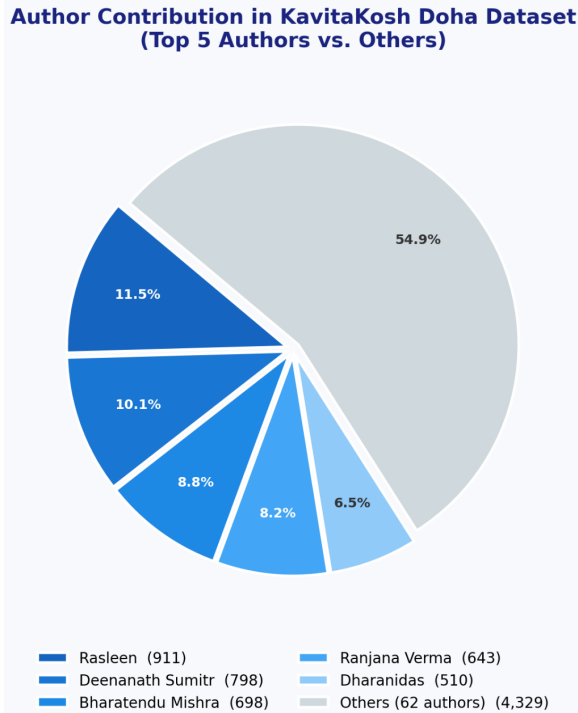


Figure 1: Author contribution: top 5 poets vs. 62 other contributors.

the corpus. The median sentence length is 20 words, indicating considerable variability in poetic structure and phrasing conventions across different poets and literary traditions.

4 Proposed Methodology and Architecture

The proposed system generates theme-based Hindi dohas in Devanagari script while adhering to the formal constraints of the doha poetic form. Because massively multilingual language models devote vast parameter budgets to non-Indic languages and often employ tokenizers that destructively segment classical Hindi script, our pipeline trains a custom, lightweight Transformer architecture (14.7M parameters) entirely from scratch. This mitigates catastrophic forgetting and prevents overfitting on the limited doha dataset. The pipeline follows a three-stage strategy:

4.1 Custom Model Backbone and Tokenizer

We employ a completely custom Encoder-Decoder Transformer architecture. A SentencePiece BPE tokenizer is trained directly on the 58,350-poem Hindi Kavita corpus with a vocabulary of 8,000 tokens. This strategy

produces far fewer subword tokens per Hindi word than multilingual tokenizers while avoiding catastrophic fragmentation of Devanagari vowel marks (matras). One hundred sentinel tokens (`<extra_id_0>` ... `<extra_id_99>`) are reserved for the span-corruption objective, and the hard-stop token is included in the vocabulary as an explicit doha terminator.

The model architecture is kept deliberately small to prevent overfitting on the 7,889-sample doha corpus. The key hyperparameters are: embedding dimension 256, feedforward dimension 1024, 8 attention heads, 4 encoder layers, 4 decoder layers per decoder head, dropout 0.1 (Stage 1) / 0.25 (Stage 2), and a maximum sequence length of 256 tokens, yielding a total of ≈ 14.69 M trainable parameters.

4.2 Stage 1: Domain Adaptation via Span Corruption

The model is first pre-trained on the Kavita dataset using a T5-style span-corruption objective (Figure 3). Given a tokenised poem $p = (w_1, \dots, w_N)$, roughly 15% of tokens are grouped into contiguous spans of ≈ 3 tokens and replaced with unique sentinel placeholders, forming a corrupted input $\tilde{p}$. The decoder is trained to reconstruct only these masked spans under cross-entropy loss, predicting each sentinel followed by its original tokens. Since the masked spans belong to classical Hindi poetry, the model implicitly learns meter, rhyme, and long-range poetic dependencies without any labelled data.

4.3 Stage 2: Supervised Fine-tuning via Dual Decoder

The Stage 2 model replaces the single decoder with a **Unified Encoder + Dual Decoder** architecture. Figure 4 illustrates this architecture.

Input Encoding. The encoder input is formatted as a structured prompt:

`<theme> [T] </theme> <context> [C] </context>`

During training, the annotated meaning is additionally appended to the encoder input 50% of the time (*meaning dropout*), so the model learns to generate the doha both with and without prior access to the plain-language explanation.

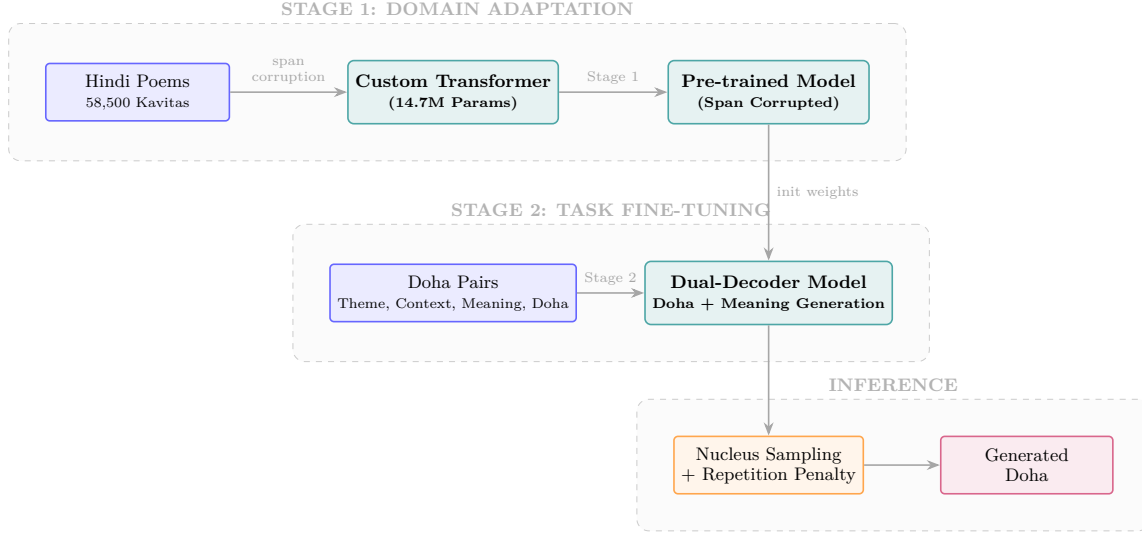


Figure 2: Overall pipeline. Stage 1 pre-trains a custom 14.7M-parameter Transformer via T5-style span corruption on a large Hindi poetry corpus. Stage 2 fine-tunes the model using a dual-decoder layout to simultaneously generate a plain-language meaning and the structured doha. Inference uses nucleus sampling and hard-stop formatting constraints.

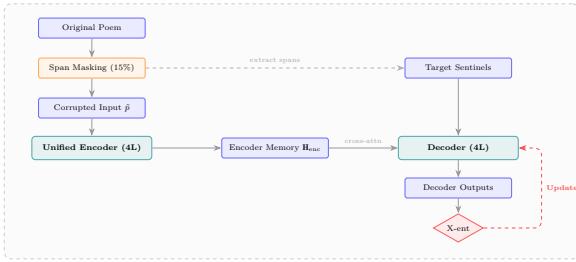


Figure 3: Stage 1: Span Corruption Pre-training. 15% of input tokens are replaced by sentinel tokens in the encoder input; the decoder is trained to predict the dropped spans autoregressively.

Meaning Decoder: The Meaning Decoder is a standard 4-layer Transformer decoder that attends to $\mathbf{H}_{\text{enc}}$ via cross-attention and autoregressively predicts the plain-language meaning up to 60 tokens.

Forcing the model to decode an explicit semantic roadmap first acts as an intrinsic regulariser, anchoring the subsequent doha to the correct thematic content. The decoder’s final hidden states form the meaning memory matrix $\mathbf{H}_{\text{mean}}$.

Doha Decoder and Gated Cross-Attention: The Doha Decoder is responsible for generating the highly constrained verse (up to 48 tokens). Each `DohaDecoderLayer` computes two independent cross-attention streams and fuses them with a per-layer learnable scalar gate g (initialised at 0.5):

$$\mathbf{C} = \sigma(g) \cdot \text{Attn}(\mathbf{X}, \mathbf{H}_{\text{enc}}) + (1 - \sigma(g)) \cdot \text{Attn}(\mathbf{X}, \mathbf{H}_{\text{mean}}) \quad (1)$$

This allows each decoder layer to learn independently how much to rely on the original user prompt versus the generated meaning. A gate value near 1.0 indicates strong encoder dependence; near 0.0 indicates reliance on meaning-derived context.

Joint Objective and Training Dynamics: Both decoders use teacher-forced cross-entropy. The total loss is:

$$\mathcal{L}_{\text{total}} = 0.2 \cdot \mathcal{L}_{\text{mean}} + 0.8 \cdot \mathcal{L}_{\text{doha}} \quad (2)$$

The doha task is weighted four times more heavily because structural verse generation is significantly harder than meaning generation. Label smoothing of 0.15 and dropout of 0.25 guard against overfitting on the 7,889-sample doha corpus.

4.4 Inference: Two-Pass Generation

A naïve single-pass decode fails because the model was trained with the plain-language meaning present in the encoder input 50% of the time (meaning dropout). Running the encoder on only the bare theme-context prompt at test time therefore places the model out-of-distribution. We close this gap

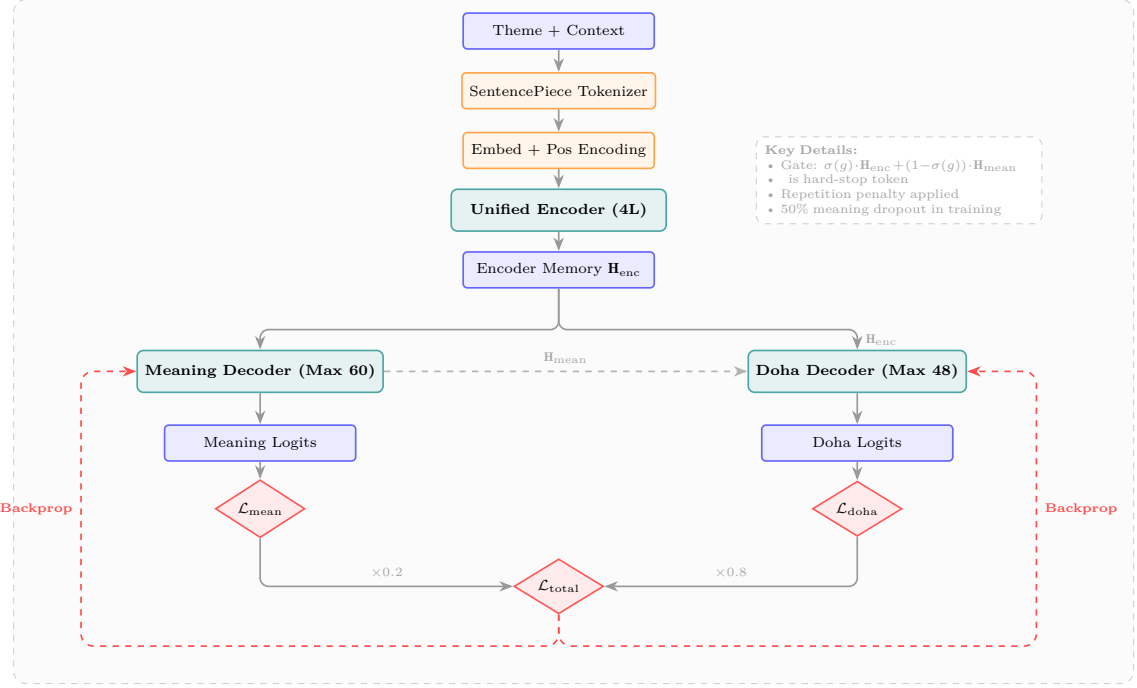


Figure 4: Stage 2: Dual-Decoder fine-tuning. The Unified Encoder processes the theme and context. The Meaning Decoder first generates a plain-language explanation ($\mathbf{H}_{\text{mean}}$), which the Doha Decoder fuses with the encoder memory via a per-layer learnable gate. Both decoders are trained jointly with a weighted cross-entropy objective.

with a **two-pass pipeline** implemented in `generate_doha()`, illustrated in Figure 5.

Pass 1: Meaning Generation. The encoder processes `<theme> [T] </theme>` `<context> [C] </context>` and the Meaning Decoder autoregressively generates up to 60 tokens using combined $\text{top-}k=50$ / nucleus ($p=0.92$) sampling at temperature $\tau=0.8$, with a repetition penalty of 1.3 over the last 20 tokens to reduce degenerate output. The full hidden-state sequence $\mathbf{H}_{\text{mean}}$ is retained for Pass 2.

Pass 2: Meaning-Conditioned Doha Generation: The decoded meaning text $\hat{m}$ is appended to the original prompt:

```
<theme> [T] </theme> <context> [C] </context>
<meaning>  $\hat{m}$  </meaning>
```

The encoder re-encodes this augmented sequence, producing $\mathbf{H}_{\text{enc2}}$ that is now conditioned on the generated semantic content. The Doha Decoder generates up to 48 tokens using the same nucleus sampling parameters but with a **stricter repetition penalty of 1.5**, since degenerate doha outputs tend to repeat short phrases or the danda separator. At each decoding step the `DohaDecoderLayer` fuses $\mathbf{H}_{\text{enc2}}$ and $\mathbf{H}_{\text{mean}}$ via the learned gate:

$$\mathbf{C} = \sigma(g) \cdot \text{Attn}(\mathbf{X}, \mathbf{H}_{\text{enc2}}) + (1 - \sigma(g)) \cdot \text{Attn}(\mathbf{X}, \mathbf{H}_{\text{mean}}) \quad (3)$$

allowing each layer to smoothly interpolate between structural cues in the re-encoded prompt and the semantic content captured in the meaning hidden states.

Hard-Stop Termination: Generation halts as soon as either the end-of-sequence token `</s>` or the Devanagari double danda is produced.

4.5 Training Setup

Both training stages were executed on Kaggle’s free-tier notebook environment using a **dual NVIDIA Tesla T4 GPU** accelerator (2×16 GB GDDR6, CUDA 11.x). Multi-GPU utilisation was implemented via PyTorch `nn.DataParallel`, which replicates the model across both devices and aggregates gradients after each backward pass. All experiments used PyTorch with `torch.cuda.amp` automatic mixed-precision (AMP) training, halving memory usage and accelerating matrix operations without loss of numerical stability.

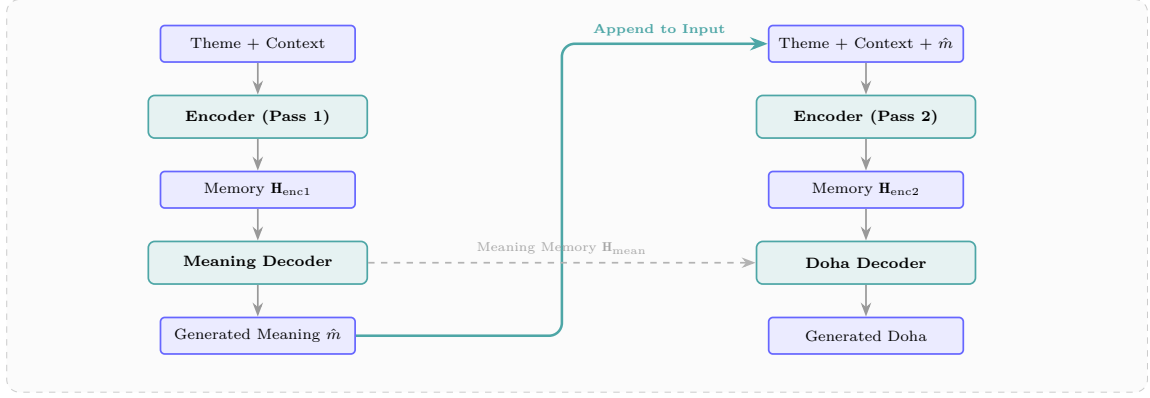


Figure 5: Two-Pass Inference Pipeline. **Pass 1:** the encoder processes only theme and context; the Meaning Decoder generates a plain-language meaning $\hat{m}$ using nucleus sampling. **Pass 2:** $\hat{m}$ is appended to the original prompt, the encoder re-encodes the augmented input producing $\mathbf{H}_{\text{enc2}}$, and the Doha Decoder attends to both $\mathbf{H}_{\text{enc2}}$ and the meaning hidden states $\mathbf{H}_{\text{mean}}$ to generate the final doha.

A global random seed of 42 was fixed across Python, NumPy, and all CUDA streams to ensure reproducibility.

Table 2: Training Hyperparameters — Stage 1 vs. Stage 2

Hyperparameter	Stage 1	Stage 2
Batch size (per GPU)	64	32
Effective batch size	128	64
Epochs	50	100 (+ early stopping)
Val split	5%	10%
Checkpoint frequency	every 5 epochs	every 10 epochs
Optimiser	AdamW	AdamW (multi-group)
β_1, β_2	0.9, 0.98	0.9, 0.98
ϵ	10^{-9}	10^{-9}
Weight decay	0.01	0.01
Gradient clipping	1.0	1.0
Base learning rate	3×10^{-4}	1×10^{-4} (frozen phase)
Encoder LR (unfrozen)	—	1×10^{-5}
Meaning Decoder LR	—	5×10^{-5}
Doha Decoder LR	—	1×10^{-4}
Minimum LR	1×10^{-6}	1×10^{-6}
LR warmup steps	500	200
LR schedule	Warmup + Cosine decay	Warmup + Cosine decay
Loss function	Cross-Entropy	Weighted Cross-Entropy
Label smoothing	0.1	0.15
Loss weights	—	$\lambda_{\text{mean}} = 0.2, \lambda_{\text{doha}} = 0.8$
Dropout	0.1	0.25
Encoder freeze epochs	—	5 (then unfrozen)
Meaning dropout prob.	—	0.5
Early stopping patience	—	20 epochs
Max encoder seq. length	256 tokens	256 tokens
Max meaning gen. length	—	60 tokens
Max doha gen. length	—	48 tokens

5 Experimental Details

We benchmark five baseline models ranging from simple statistical methods to large pre-trained transformers. The primary purpose of these experiments is to identify where off-the-shelf approaches fail on the doha generation task and to motivate each component of our custom architecture. All baselines were trained on 7,100 dohas (the remaining 789 were held out for evaluation); our custom

model was trained on the same split. A quantitative comparison across all systems is presented in Section 7.

5.1 N-Gram and LSTM Baselines

An N-gram language model with character-level tokenization serves as the simplest statistical baseline. The output is semantically incoherent, with no mechanism to track or enforce matra structure. A character-level LSTM (2-layer, 256 hidden units, dropout 0.5) produces fragments that superficially resemble doha formatting but contain non-words and no consistent thematic thread. Both models confirm that purely statistical approaches fail on all three dimensions: semantic understanding, structural validity, and vowel preservation.

LSTM output (Epoch 10): मोर पच्छान
॥ वै लाजै के प्रान कै, प्रभु करै कहि मुख दिननाइ
सुमान ॥ राम रूप भी शाल स

5.2 GPT-2 (Hindi) Model Analysis

The fourth baseline used `surajp/gpt2-hindi` (Patil, 2020), a Hindi-adapted GPT-2 model with 81.9M parameters pre-trained on Hindi web text. GPT-2 is a causal (decoder-only) language model.

Generation Results.

Output: समय की बर्बादी, कर दें पुराने कौन?
मूल मंजूर है वक्रत तो, जब लग जाएँगे शौक to
all the way, it should have been true.
a newest posts with label and ...

Two characteristic failure observed: code-switching (English text mid-generation due

to mixed-script pretraining data), and structurally incoherent continuations that drift into story-like prose after the opening lines.

5.3 IndicBART Model Analysis

IndicBART with 440M parameters, it is the largest model evaluated in our baseline study. Generation uses a language tag <2hi> appended to the input to direct the model towards Hindi output. The model was fine-tuned on 7,100 (theme + context → doha) input-output pairs for 6 epochs

Generation Results.

Input: विषय: विरह, संदर्भ: विरह की अग्नि

Output (Epoch 3): <s><2hi>विरह क
अगन क कय, कय न कय कय। कय दन क कछ
क, कछ न कछ सखय ॥सहत हय कछ ...

Several problems are immediately visible. The output retains the raw special tokens (<s>, <2hi>) indicating the model never learned to suppress them. More critically, all vowel matras are stripped from words विरह becomes वरह, अग्नि becomes अगन a result of the IndicBART tokenizer’s stripping away matras before processing the input.

5.4 mT5 Model Analysis

mT5 (Xue et al., 2021) is a multilingual encoder-decoder transformer pretrained on the mC4 corpus covering 101 languages including Hindi, making it naturally suited to conditional generation tasks. Fine-tuning mT5-base on our dataset for 5 epochs produced the most coherent output observed across all baseline models:

Input: विषय:भक्ति | संदर्भ: ईश्वर के प्रति समर्पण

Output: कल्याण की नई भक्ति, सभी के प्रति समर्पण, प्रकार माँवति का. उसके प्रति समर्पण तत्त्व को ॥

The output contains full Hindi words, thematic vocabulary related to devotion (भक्ति, समर्पण, कल्याण), and ends with the doha terminator ॥ showing the model has learned the doha format to some extent. However, the output is a single run-on line rather than two structured lines, the matra counts are not valid, and parts of the text are semantically garbled (माँवति का is not a standard Hindi phrase).

5.5 byT5 Model Analysis

byT5 (Xue et al., 2022), a tokenization-free T5 variant operating on UTF-8 bytes, was selected specifically to address the matra-stripping failure of IndicBART. Our training pipeline followed two stages: domain adaptation on 20,000 Hindi poems using span-masking and sentence-shuffling (3 epochs, lr 3×10^{-4}), followed by fine-tuning on 7,100 dohas with a differentiable matra loss.

In stage 2 with Matra Loss Integration the model was fine-tuned on 7,100 dohas.

Input: विषय: नीति | संदर्भ: सज्जन पुरुषों का स्वभाव

Output: कोई जाने स्वप्न में, | तुम रहन से सज्जन | | सुनाई के सजन मानि, | दुर्लभ कहत कोई

Evaluation: Matras [13, 11, 13, 11] .

Unlike all previous baselines, the Stage 2 byT5 model successfully produced outputs in the two-line format with correct matra counts. However, the generated text lacks semantic meaning the words are structurally placed but do not form a coherent thematic statement. This remaining semantic gap motivates our dual-decoder architecture: byT5 can satisfy the structural constraint but cannot jointly satisfy semantic coherence without an explicit meaning-conditioning mechanism.

6 Evaluation Metrics

To evaluate the quality of the generated dohas, we employ a combination of structural and novelty checks.

6.1 Matra Accuracy Score (MAS)

Hindi prosody is based on *matras* units of syllabic duration rather than stress. Each character in Devanagari contributes a fixed number of matras: short vowel sounds (*laghu*) contribute 1 matra, and long vowel sounds (*guru*) contribute 2 matras. Consonants in conjunct clusters and half-characters follow additional rules. A valid doha has four *charans* (quarter-lines) with the pattern **13 + 11 | 13 + 11**, totalling 48 matras.

6.1.1 Matra Accuracy Score Formula

For a generated doha with charan matra counts m_1, m_2, m_3, m_4 , MAS is the total absolute deviation from the ideal pattern:

Table 3: Matra Weights in Hindi Prosody (Chhand Shastra)

Character Category	Weight	Rule Description	Example
Hrasva (Short)	1	Includes short vowels (अ, इ, उ, ऋ) and their matras.	क, कि, कु
Dirgha (Long)	2	Includes long vowels (आ, ई, ऊ, ए, ऐ, औ) and matras.	का, की, के
Anusvara (◌ं)	2	Any letter followed by a dot (bindu) becomes Guru.	सं (sam)
Visarga (◌ः)	2	Any letter followed by two dots becomes Guru.	तः (tah)
Nukta (◌ः)	0	The dot below does not alter the base matra weight.	फ़, ज़
Pre-Samyukt	2	A short letter followed by a half-consonant becomes Guru.	सं in 'सत्य'
Chandrabinidu (◌ँ)	1	Nasalization without stress does not increase weight.	हँस (हँ = 1)

$$MAS = |m_1 - 13| + |m_2 - 11| + |m_3 - 13| + |m_4 - 11| \quad (4)$$

A perfectly valid doha scores $MAS = 0$. Unlike a binary valid/invalid flag, MAS is a continuous metric it quantifies *how far* each output is from correctness, making it suitable for comparing models that are all structurally imperfect to varying degrees.

6.1.2 Evaluation on Scraped Dataset

Running the matra counter on our scraped dataset of 7,889 dohas revealed important insights about real-world doha text quality, summarized in Table 4.

Table 4: Matra Counter Results on Scraped Dataset

Metric	Value
Total dohas evaluated	7,889
Dohas with all 4 charans parsed	6326 (80.2%)
Dohas collapsed to 2 charans	1243 (15.8%)
Flagged valid (48 total matras)	5257 (66.6%)

Out of the full dataset of **7,889 dohas**, **7,405 dohas (93.9%)** fall within a ± 2 matra tolerance of the ideal **48-matra** target, with only **484 dohas (6.1%)** deviating beyond this range confirming that the scraped corpus is of high prosodic quality.

6.2 Novelty Score (NS)

Novelty is measured using Self-BLEU to quantify diversity among generated dohas. For a set $\{D_1, D_2, \dots, D_n\}$:

$$NS = 1 - \frac{1}{n} \sum_{i=1}^n BLEU(D_i, \{D_j\}_{j \neq i}) \quad (5)$$

Higher values indicate greater diversity. This guards against degenerate models that repeatedly produce the same output or closely paraphrase training examples.

6.3 Human Evaluation Metrics

- **Grammatical Correctness (GC):** Evaluates the linguistic accuracy of the generated Hindi text. It measures adherence to syntax, morphology, and standard grammatical rules.

$$\text{Score} \in \{1, 2, 3, 4, 5\}$$

- **Thematic Coherence (TC):** Assesses the semantic relevance of the Doha to the user-provided theme and context. It determines if the output logically follows the input prompt.
- **Poetic Soundness (PS):** A structural metric measuring the rhythmic flow and prosodic accuracy. This checks for the typical Doha meter and the end-rhyme (*Tukant*) consistency.

7 Analysis of Results

7.1 Baseline Comparison

Table 5 summarises structural and qualitative performance across all six systems. The clear progression from purely statistical models (N-gram, LSTM) through large pretrained models (GPT-2, IndicBART, mT5) to structurally-aware models (byT5, ours) validates our hypothesis that standard generative models cannot satisfy the 13+11+13+11 prosodic constraint without explicit structural supervision, regardless of model scale.

Table 5: Baseline Model Comparison

Model	Valid Struct.	Matra Correct	Semantic Coherence	Vowel Preserved
N-gram	No	No	No	N/A
LSTM	No	No	No	Moderate
GPT-2 (Hindi)	No	No	Partial	Yes (code-switch)
IndicBART	No	No	No	No (stripped)
mT5-base	No	No	Partial	Yes
byT5	Partial	Partial	No	Yes
Ours	Partial	Partial	Yes	Yes

7.2 Automated Evaluation

Our custom Transformer model demonstrates a sophisticated command over the structural

and thematic nuances of Hindi prosody. Quantitatively, the generated corpus of 50 dohas across 50 different themes and contexts was evaluated on four key metrics, summarized in Table 6.

Table 6: Quantitative Evaluation Metrics

Metric	Score
Average Matra Accuracy Score	4.91
Average Matra Count	46.58
External Novelty (NS)	0.9257
Internal Novelty (Self-BLEU)	0.9729

While the observed average Matra Accuracy Score (MAS) of **4.91** deviates from the theoretical **0-error** ideal, it is a very strict constraint to follow. On running the MAS script on our dataset as well we found an average MAS of 4.5 so we are not that off. We can see though the MAS is around 5 the average matra score is off by just **1.42** which suggests that the dohas all in all are following the structure quite well. Furthermore, the model exhibits exceptional creative independence, as confirmed by the high novelty scores. These metrics confirm that the model is not merely retrieving sequences from the training data, but is instead performing genuine linguistic synthesis and responding distinctly to different themes such as **भक्ति** (devotion) or **श्रृंगार** (beauty).

7.3 Human Evaluation Results

To semantically assess the quality of the generated dohas beyond automated metrics, we conducted a human evaluation via Google Forms distributed to a small group of evaluators. A total of 50 generated dohas were evaluated, distributed across 10 separate forms with 5 dohas per form, each paired with its corresponding theme and context. We got 25 responses in total.

Table 7: Human Evaluation Results

Metric	Average Score (out of 5)
Grammatical Correctness (GC)	3.61
Poetic Soundness (PS)	3.58
Thematic Coherence	3.53

From responses we observed the relative ordering : **GC > PS > Coherence** is consistent with the expectation that a transformer pre-trained on a large Hindi poetry cor-

pus would internalize surface-level grammatical and rhythmic patterns more readily than deeper semantic alignment with a given theme and context. The narrow spread (range 0.08) suggests balanced output quality rather than trade-offs between dimensions.

In terms of per-doha performance, the two highest rated dohas (with at least two responses) are:

Doha-1: दीपो दीपक जलता सदा, जले रात दिन रात। अँधियारे से सूर्य से, जगमगा उजियार ॥

Theme: साहस *Context:* अंधेरी सुरंग के अंत में प्रकाश की उम्मीद

GC = 4.50 COHERENCE = 4.00 PS = 4.50

Doha-2: कोयल की वाणी करे, मन में रस समान। सुरभित करती है सदा, मीठे-मन की मुस्कान ॥

Theme: वाणी *Context:* कोयल के स्वर

GC = 4.00 COHERENCE = 4.50 PS = 4.50

The two lowest rated dohas (with at least two responses) are:

Doha-1: कबीर नीरस की नदी, बिन मन को धार। बिना नेह पिया न हो कभी, प्यास नहीं आस ॥

Theme: विरह *Context:* जुदाई का दुख

GC = 2.00 COHERENCE = 2.00 PS = 2.00

Doha-2: धरनी गुन गुप्त है, रहनि एक संसार। परमात्म जग में गनत, आचार रूप ॥

Theme: आध्यात्म *Context:* परमात्मा का स्वरूप

GC = 2.00 COHERENCE = 2.33 PS = 1.67

Data and Code Availability

To facilitate reproducibility, all resources developed for this project have been made publicly available:

- **Doha Dataset:** <https://www.kaggle.com/datasets/kgan31/doha-simple-context/>

- **Poems Dataset:** <https://www.kaggle.com/datasets/kgan31/complete-kavita-dataset/data>
- **Custom Transformer Checkpoint:** https://huggingface.co/nikpatidar333/doha-generation-model_v2
- **Fine-Tuned byT5 Checkpoint:** https://huggingface.co/KGan31/Doha-Gen-Stage2_Matra_loss_integrated/

8 Limitations

Some of the limitations should be noted. First, the Gemini-generated theme, context, and meaning annotations have not been manually verified; minor labelling inconsistencies in the training data may reduce alignment between the semantic roadmap and generated dohas. Second, the human evaluation used a small convenience sample, limiting the statistical reliability of the reported scores. Third, the model’s performance on rare themes (under-represented in the training corpus) is unknown and likely lower than the reported averages.

9 Conclusion

We presented a system for constrained Hindi doha generation combining a 14.7M-parameter custom Transformer, two-stage training with span-corruption pre-training and dual-decoder fine-tuning, and a two-pass inference strategy. Experiments confirm that standard generative models regardless of scale cannot satisfy the 13+11+13+11 prosodic constraint without explicit structural supervision. Our model achieves an average matra count of 46.58/48, External Novelty of 0.9257, and an average human evaluation score of 3.57/5, demonstrating that lightweight domain-specific architectures can effectively address constrained generation in low-resource Indic poetry.

10 Future Work

Future work should address the identified limitations: incorporating formal matra-constrained decoding (hard vocabulary masking based on real-time charan matra tracking), augmenting low-resource themes through

back-translation or paraphrase, replacing Gemini-generated annotations with expert human annotations, and conducting a larger-scale human evaluation with measured inter-annotator agreement.

References

- Edwin Agnew, Michelle Qiu, Lily Zhu, Sam Wiseman, and Cynthia Rudin. 2023. [The mechanical bard: An interpretable machine learning approach to Shakespearean sonnet generation](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1627–1638, Toronto, Canada. Association for Computational Linguistics.
- Zhiyuan Hu, Chumin Liu, Yue Feng, Anh Tuan Luu, and Bryan Hooi. 2023. [Poetrydiffusion: Towards joint semantic and metrical manipulation in poetry generation](#). *Preprint*, arXiv:2306.08456.
- Manoj Balaji Jagadeeshan, Samarth Bhatia, Pre-tam Ray, Harshul Surana, P AkhilRajeev, Priya Mishra, Annarao Kulkarni, Ganesh Ramakrishnan, AP Prathosh, and Pawan Goyal. 2025. [Chandomitra: Towards generating structured sanskrit poetry from natural language inputs](#).
- KavitaKosh. [Kavitakosh: A repository of hindi poetry](#).
- Piji Li, Haisong Zhang, Xiaojiang Liu, and Shuming Shi. 2020. [Rigid formats controlled text generation](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 742–751, Online. Association for Computational Linguistics.
- Suraj Patil. 2020. [Gpt-2 hindi](#).
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. [Byt5: Towards a token-free future with pre-trained byte-to-byte models](#). *Preprint*, arXiv:2105.13626.
- Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. [mt5: A massively multilingual pre-trained text-to-text transformer](#). *Preprint*, arXiv:2010.11934.